

Experiment No. 3

Aim:

To create a **Cryptocurrency using Python** and perform **mining in the Blockchain** created using a simulated **three-node P2P blockchain network**.

Theory:

1. Blockchain

A **blockchain** is a distributed and decentralized ledger that stores information in a series of linked blocks. Each block contains transaction data, a timestamp, the previous block's hash, and its own hash. Since blocks are linked using hashes, modifying a previous block affects the subsequent blocks.

2. Mining

Mining is the process of collecting pending transactions into a block and solving a computational puzzle using **Proof-of-Work (PoW)**. Once a valid proof is found, the block is added to the blockchain and broadcast to connected peers. The miner receives a cryptocurrency reward for successfully mining the block.

3. P2P Blockchain Network

In this experiment, three blockchain nodes are used:

Node 1 → Port 5001

Node 2 → Port 5002

Node 3 → Port 5003

Each node maintains its own copy of the blockchain and communicates with other nodes to share and validate blocks.

4. Transactions

A transaction represents the transfer of cryptocurrency from one user to another. Each transaction contains:

- **Sender**
- **Receiver**
- **Amount**

Pending transactions are collected and added to a block during mining. A mining reward transaction is also added for the miner.

5. Proof-of-Work

Proof-of-Work requires the miner to perform computational calculations to find a valid proof. In the provided implementation, the hash generated using SHA-256 must satisfy the required condition before the block can be mined.

6. Longest Chain Rule

If different nodes have different versions of the blockchain, the longest valid chain is selected. The `/replace_chain` function obtains chains from connected peers and replaces the current

Experiment No. 3

chain if a longer valid chain is found. This keeps the blockchain synchronized across the network.

7. Tools and Libraries

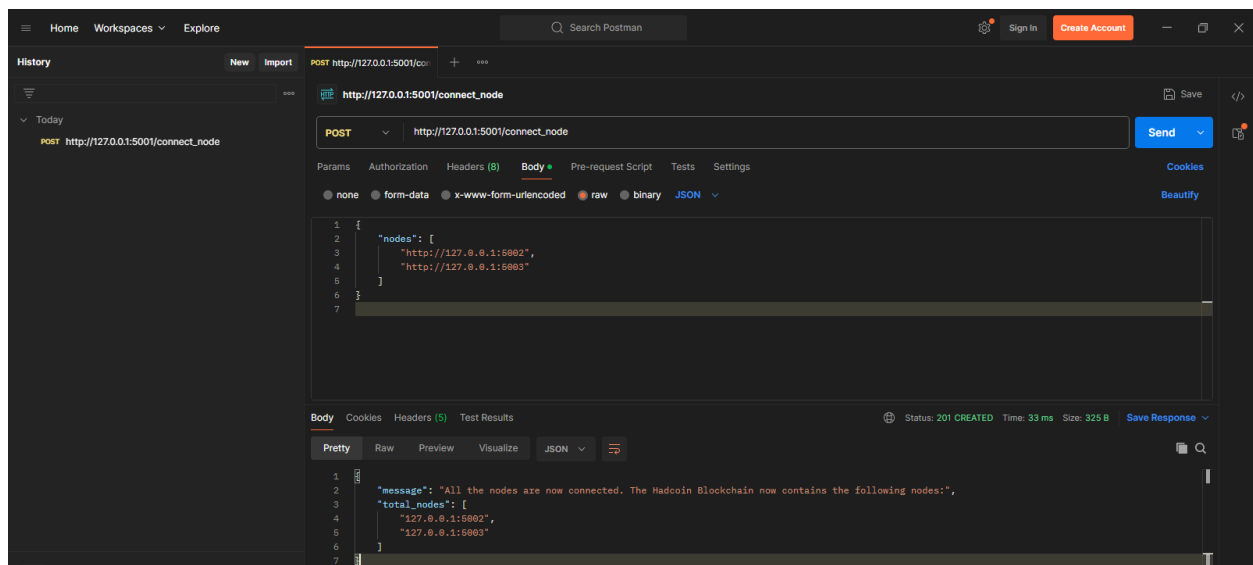
The experiment uses:

- **Python** - Programming language
- **Flask** - Creates blockchain API endpoints
- **Postman** - Sends and tests API requests
- **Requests** - Communication between blockchain nodes
- **Hashlib** - SHA-256 hashing
- **Datetime** - Timestamp generation
- **UUID** - Generates unique identifiers
- **URL parsing/request libraries** - Handling network requests

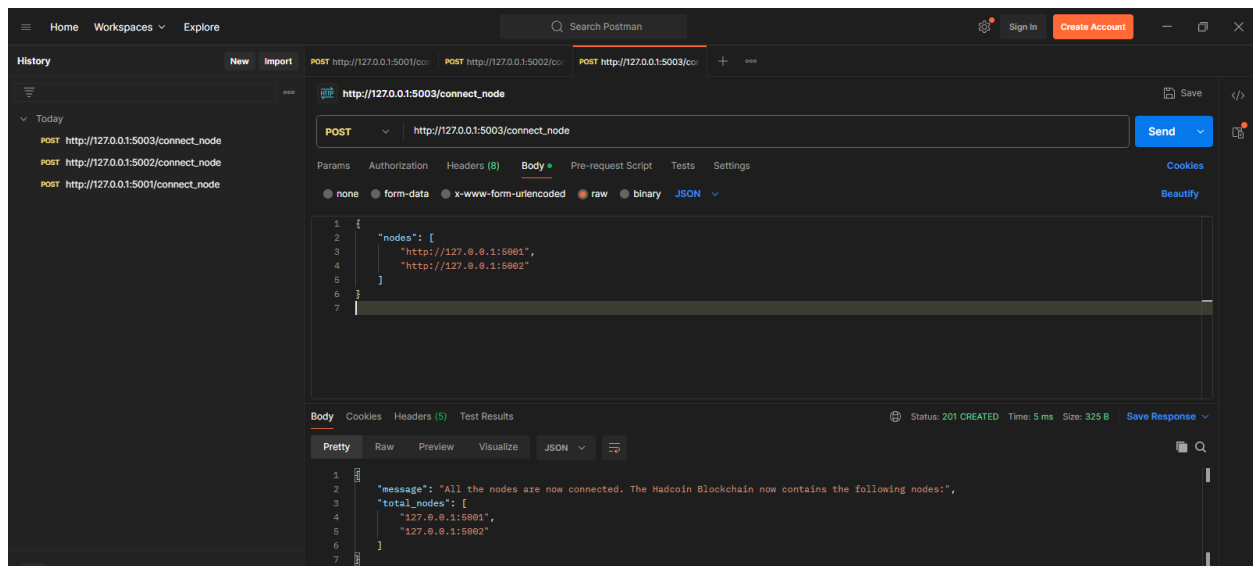
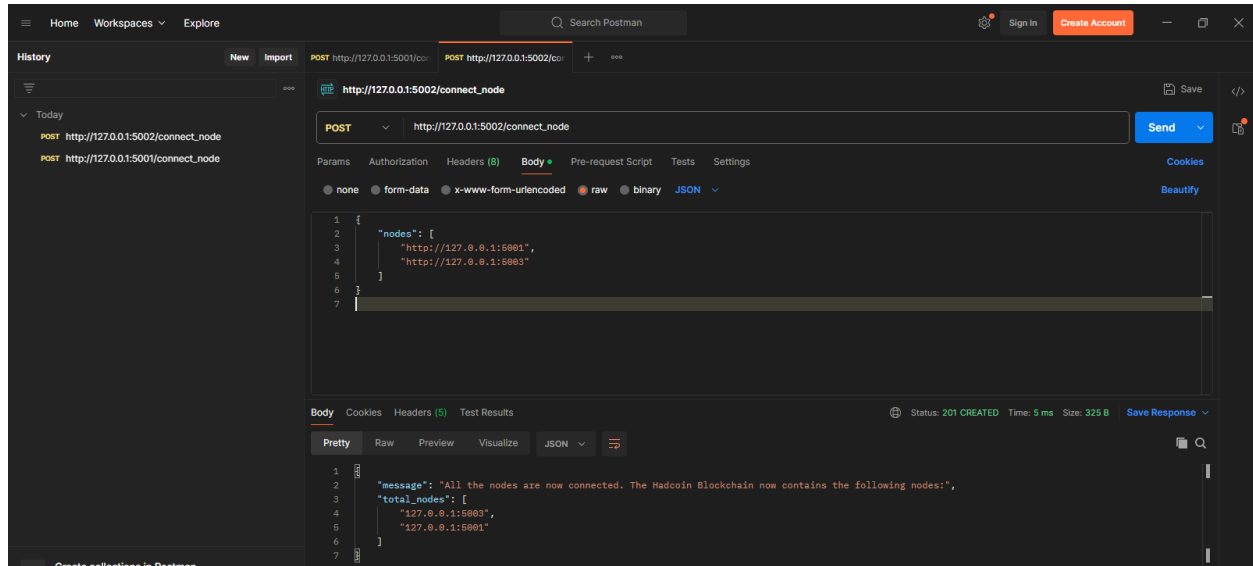
These tools and libraries are listed in the provided experiment material.

Output:

1. Nodes Connected :

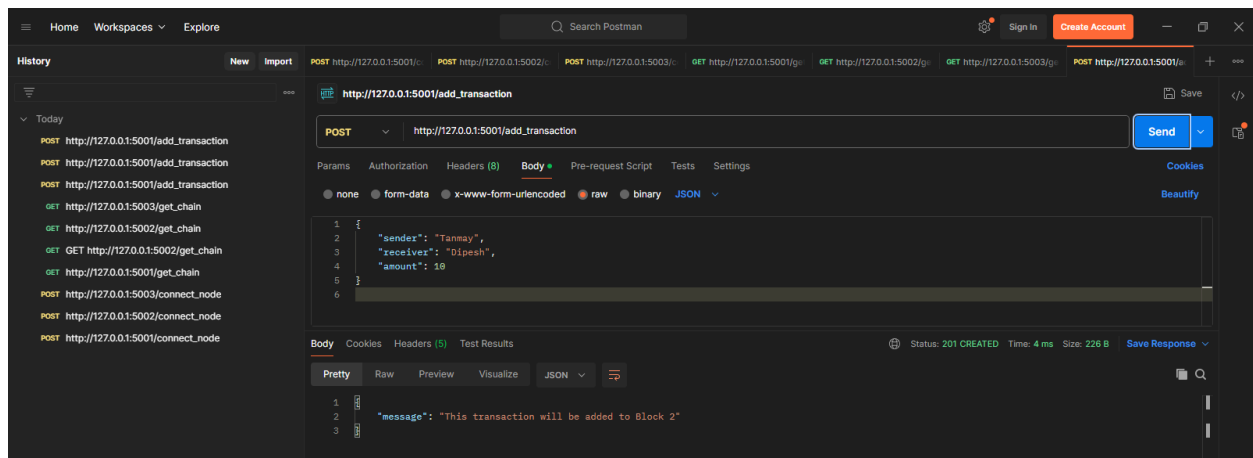
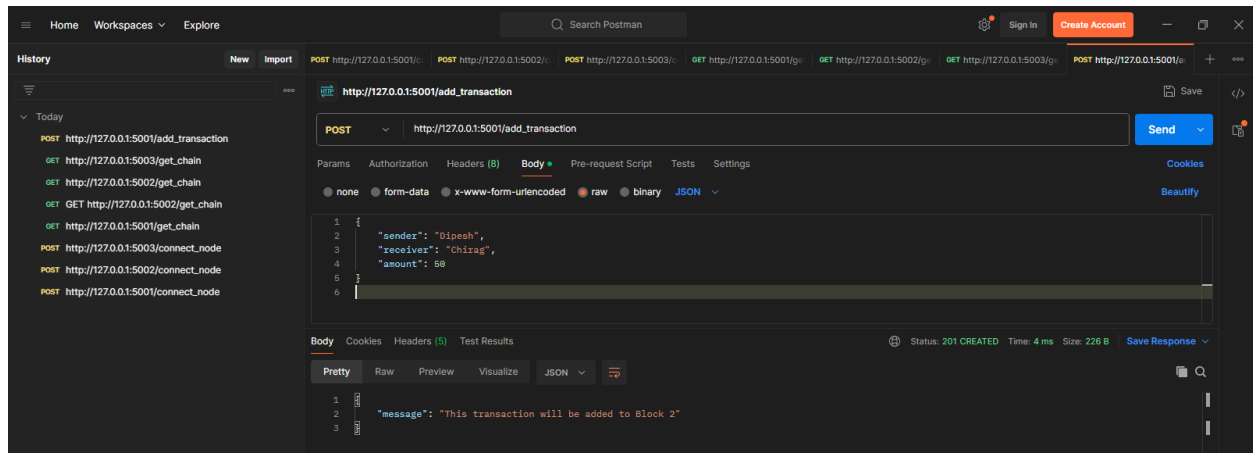


Experiment No. 3



Experiment No. 3

2. Transactions Added :



Experiment No. 3

3. Block Mined :

History: GET http://127.0.0.1:5001/mine_block, POST http://127.0.0.1:5001/add_transaction, POST http://127.0.0.1:5001/add_transaction, POST http://127.0.0.1:5001/add_transaction, GET http://127.0.0.1:5003/get_chain, GET http://127.0.0.1:5002/get_chain, GET http://127.0.0.1:5002/get_chain, GET http://127.0.0.1:5001/get_chain, POST http://127.0.0.1:5003/connect_node, POST http://127.0.0.1:5002/connect_node, POST http://127.0.0.1:5001/connect_node

Request: GET http://127.0.0.1:5001/mine_block

Params: Authorization, Headers (6), Body, Pre-request Script, Tests, Settings

Query Params:

Key	Value
Key	Value

Body: Status: 200 OK, Time: 4 ms, Size: 618 B

```
1 {
2   "index": 2,
3   "message": "Congratulations, you just mined a block!",
4   "previous_hash": "464839b7d87bba2369b9e3e637e9f09b1e8a233a848be8a6f9f2e0b23207fa2",
5   "proof": 533,
6   "timestamp": "2026-10-07 12:44:13.816282",
7   "transactions": [
8     {
9       "amount": 50,
10      "receiver": "Chirag",
11      "sender": "Dipesh"
12    },
13    {
14      "amount": 20,
15      "receiver": "Tanmay",
16      "sender": "Chirag"
17    },
18    {
19      "amount": 10,
20      "receiver": "Dipesh",
21      "sender": "Tanmay"
22    }
23  ]
24 }
```

History: GET http://127.0.0.1:5001/mine_block, POST http://127.0.0.1:5001/add_transaction, POST http://127.0.0.1:5001/add_transaction, POST http://127.0.0.1:5001/add_transaction, GET http://127.0.0.1:5003/get_chain, GET http://127.0.0.1:5002/get_chain, GET http://127.0.0.1:5002/get_chain, GET http://127.0.0.1:5001/get_chain, POST http://127.0.0.1:5003/connect_node, POST http://127.0.0.1:5002/connect_node, POST http://127.0.0.1:5001/connect_node

Request: GET http://127.0.0.1:5001/mine_block

Params: Authorization, Headers (6), Body, Pre-request Script, Tests, Settings

Query Params:

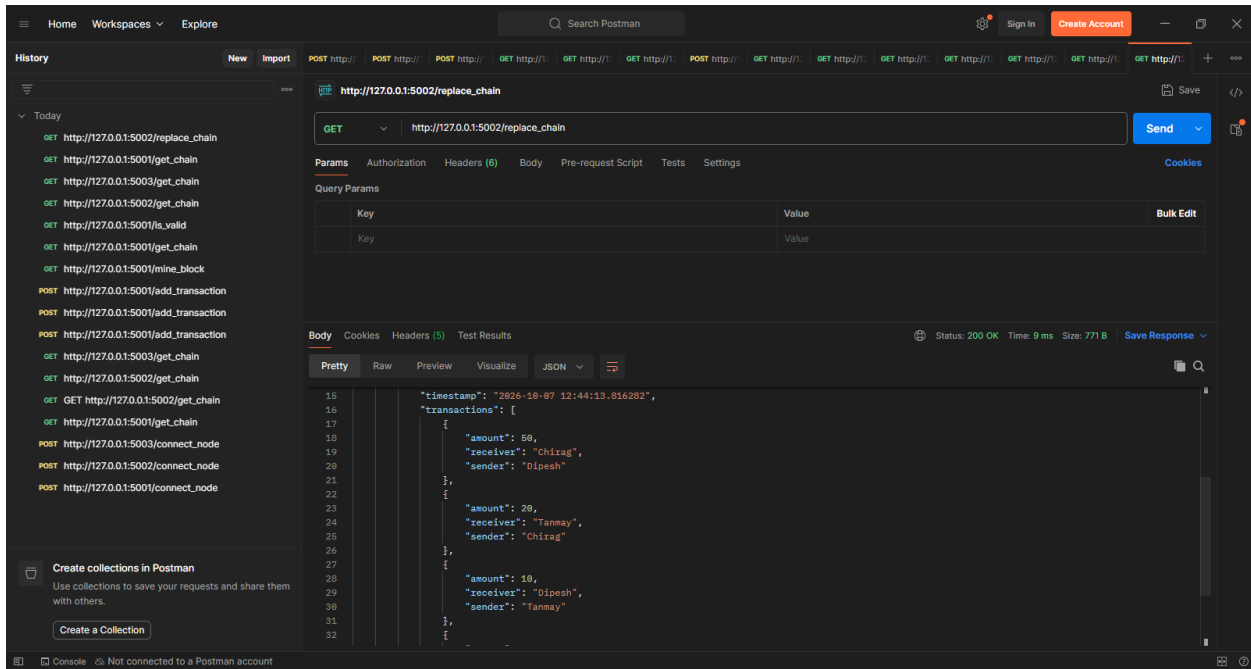
Key	Value
Key	Value

Body: Status: 200 OK, Time: 4 ms, Size: 618 B

```
9 {
10   "amount": 50,
11   "receiver": "Chirag",
12   "sender": "Dipesh"
13 },
14 {
15   "amount": 20,
16   "receiver": "Tanmay",
17   "sender": "Chirag"
18 },
19 {
20   "amount": 10,
21   "receiver": "Dipesh",
22   "sender": "Tanmay"
23 },
24 {
25   "amount": 1,
26   "receiver": "Richard",
27   "sender": "7f64508161ee4a5ba94ea59e1266f267"
28 }
29 }
```

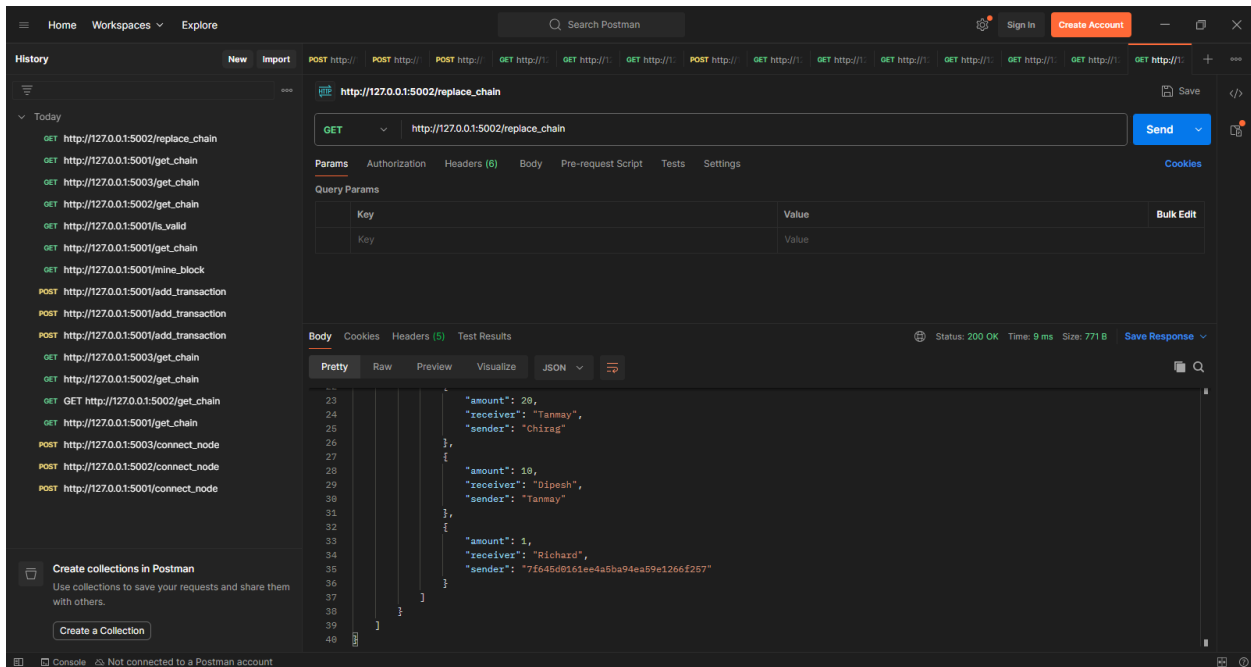
Experiment No. 3

4. Chain Replacement :



Postman interface showing a GET request to `http://127.0.0.1:5002/replace_chain`. The response body is a JSON array of transactions.

```
{
  "timestamp": "2026-10-07 12:44:13.816282",
  "transactions": [
    {
      "amount": 50,
      "receiver": "Chirag",
      "sender": "Dipesh"
    },
    {
      "amount": 20,
      "receiver": "Tanmay",
      "sender": "Chirag"
    },
    {
      "amount": 10,
      "receiver": "Dipesh",
      "sender": "Tanmay"
    }
  ]
}
```



Postman interface showing a GET request to `http://127.0.0.1:5002/replace_chain`. The response body is a JSON array of transactions.

```
{
  "amount": 20,
  "receiver": "Tanmay",
  "sender": "Chirag"
},
{
  "amount": 10,
  "receiver": "Dipesh",
  "sender": "Tanmay"
},
{
  "amount": 1,
  "receiver": "Richard",
  "sender": "7f645d8161ee4a8ba94ea59e1266f257"
}
]
```

Experiment No. 3

The screenshot displays the Postman application interface. On the left, the 'History' panel lists several API requests. The main workspace shows a GET request to `http://127.0.0.1:5003/replace_chain`. The 'Body' tab is selected, showing a JSON response with the following structure:

```
1 {
2   "message": "The nodes had different chains so the chain was replaced by the longest one.",
3   "new_chain": [
4     {
5       "index": 1,
6       "previous_hash": "0",
7       "proof": 1,
8       "timestamp": "2026-10-07 12:34:24.496691",
9       "transactions": []
10    },
11    {
12       "index": 2,
13       "previous_hash": "464539b7d87bba2859b993e53769f89b1e86233a848be8a6fd9f2e0b23207fa2",
14       "proof": 633,
15       "timestamp": "2026-10-07 12:44:13.816202",
16       "transactions": [
17         {
18           "amount": 50,
19         }
20       ]
21     }
22   ]
23 }
```

The status bar at the bottom indicates a successful response with status 200 OK, a time of 13 ms, and a size of 771 B. A console message at the bottom states: 'Not connected to a Postman account'.

Experiment No. 3

History: Today

- GET http://127.0.0.1:5003/replace_chain
- GET http://127.0.0.1:5002/replace_chain
- GET http://127.0.0.1:5001/get_chain
- GET http://127.0.0.1:5003/get_chain
- GET http://127.0.0.1:5002/get_chain
- GET http://127.0.0.1:5001/is_valid
- GET http://127.0.0.1:5001/get_chain
- GET http://127.0.0.1:5001/mine_block
- POST http://127.0.0.1:5001/add_transaction
- POST http://127.0.0.1:5001/add_transaction
- POST http://127.0.0.1:5001/add_transaction
- GET http://127.0.0.1:5003/get_chain
- GET http://127.0.0.1:5002/get_chain
- GET http://127.0.0.1:5001/get_chain
- POST http://127.0.0.1:5003/connect_node
- POST http://127.0.0.1:5002/connect_node
- POST http://127.0.0.1:5001/connect_node

Create collections in Postman

Use collections to save your requests and share them with others.

Create a Collection

GET http://127.0.0.1:5003/replace_chain

Params: Authorization Headers (6) Body Pre-request Script Tests Settings Cookies

Query Params

Key	Value
Key	Value

Body Cookies Headers (5) Test Results Status: 200 OK Time: 13 ms Size: 771 B Save Response

Pretty Raw Preview Visualize JSON

```
15 {
16   "timestamp": "2026-10-07 12:44:13.816282",
17   "transactions": [
18     {
19       "amount": 50,
20       "receiver": "Chirag",
21       "sender": "Dipesh"
22     },
23     {
24       "amount": 20,
25       "receiver": "Tanmay",
26       "sender": "Chirag"
27     },
28     {
29       "amount": 10,
30       "receiver": "Dipesh",
31       "sender": "Tanmay"
32     }
33   ]
34 }
```

History: Today

- GET http://127.0.0.1:5003/replace_chain
- GET http://127.0.0.1:5002/replace_chain
- GET http://127.0.0.1:5001/get_chain
- GET http://127.0.0.1:5003/get_chain
- GET http://127.0.0.1:5002/get_chain
- GET http://127.0.0.1:5001/is_valid
- GET http://127.0.0.1:5001/get_chain
- GET http://127.0.0.1:5001/mine_block
- POST http://127.0.0.1:5001/add_transaction
- POST http://127.0.0.1:5001/add_transaction
- POST http://127.0.0.1:5001/add_transaction
- GET http://127.0.0.1:5003/get_chain
- GET http://127.0.0.1:5002/get_chain
- GET http://127.0.0.1:5001/get_chain
- POST http://127.0.0.1:5003/connect_node
- POST http://127.0.0.1:5002/connect_node
- POST http://127.0.0.1:5001/connect_node

Create collections in Postman

Use collections to save your requests and share them with others.

Create a Collection

GET http://127.0.0.1:5003/replace_chain

Params: Authorization Headers (6) Body Pre-request Script Tests Settings Cookies

Query Params

Key	Value
Key	Value

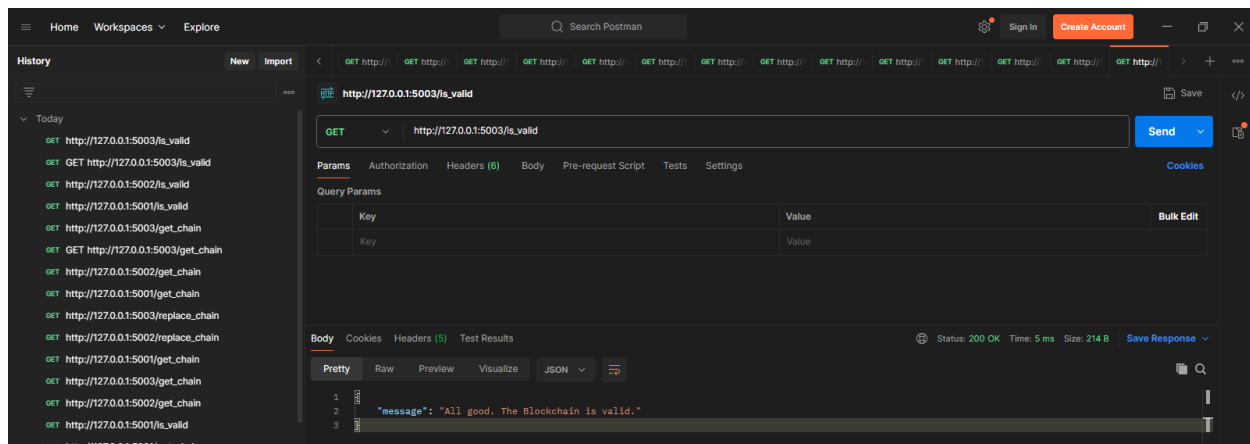
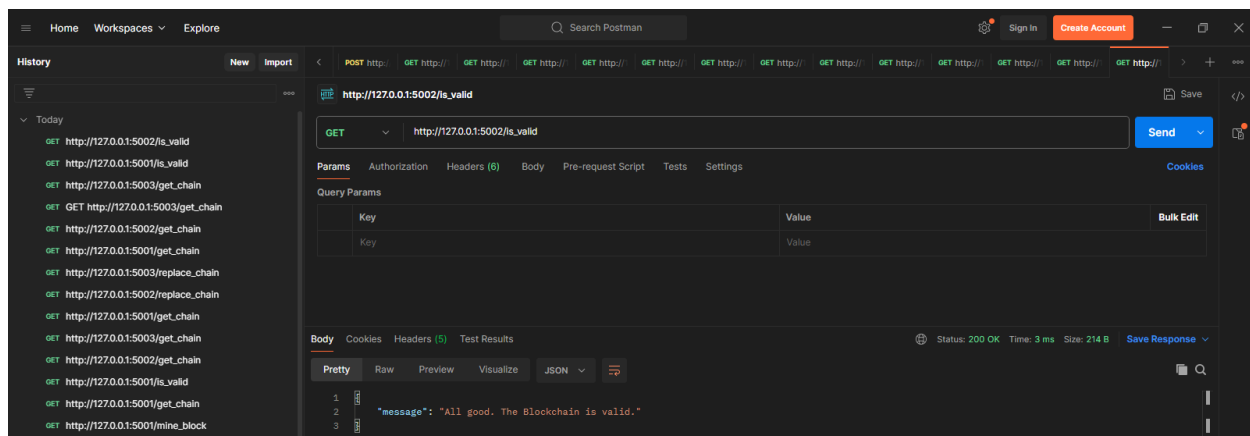
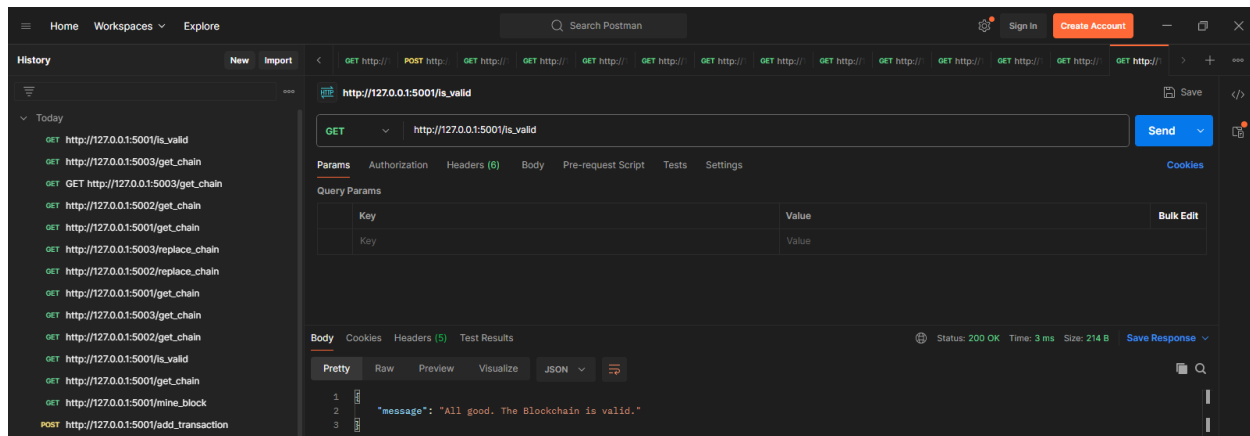
Body Cookies Headers (5) Test Results Status: 200 OK Time: 13 ms Size: 771 B Save Response

Pretty Raw Preview Visualize JSON

```
23 {
24   "amount": 20,
25   "receiver": "Tanmay",
26   "sender": "Chirag"
27 },
28 {
29   "amount": 10,
30   "receiver": "Dipesh",
31   "sender": "Tanmay"
32 },
33 {
34   "amount": 1,
35   "receiver": "Richard",
36   "sender": "77f6450b161ee4a8ba94ea69e1266f257"
37 }
38 ]
39 }
40 }
```

Experiment No. 3

5. Blockchain Validation :



CONCLUSION:

A cryptocurrency was successfully implemented using Python, Flask and a three-node P2P blockchain network. Transactions were added and mined using Proof-of-Work, and the blockchain was synchronized between nodes using the Longest Chain Rule. This experiment demonstrated cryptocurrency transactions, mining, decentralized consensus and blockchain network synchronization.